

Universidad Autónoma de Madrid
Escuela Politécnica Superior
Análisis y Diseño de Software 2023-2024
Práctica 1: Introducción a Java

Inicio: A partir del 5 de febrero.

Duración: 1 semana.

Entrega: Una hora antes del comienzo de la siguiente práctica.

Peso de la práctica: 5%

El objetivo de esta práctica es aprender el funcionamiento de algunas de las herramientas del Java Development Toolkit (JDK), comprender el esquema de funcionamiento de la máquina virtual de Java y escribir tus primeros programas en Java.

Apartado 1: Hola Mundo

Con un editor de texto, teclea el siguiente programa, y guárdalo con el nombre *HolaMundo.java*

```
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("Hola mundo!");    // muestra el string por stdout
    }
}
```

En la línea de comandos, ejecuta la siguiente sentencia:

```
javac HolaMundo.java
```

para compilar la clase *HolaMundo* y generar el fichero *HolaMundo.class* correspondiente. Ejecuta el fichero *.class* mediante la sentencia:

```
java HolaMundo
```

Nota: el nombre del fichero *.java* tiene que ser el mismo que la clase que contiene respetando mayúsculas y minúsculas. Asegúrate de que los programas *javac* y *java* están en el PATH¹.

Apartado 2: Generación de Documentación.

El programa *javadoc* permite generar documentación HTML de los distintos programas fuente leyendo los comentarios del código. Por ejemplo, modifica el programa anterior incluyendo los siguientes comentarios, añadiendo tu nombre como autor:

```
/**
 * Esta aplicación muestra el mensaje "Hola mundo!" por pantalla
 *
 * @author Estudiante EPS estudiante.eps@uam.es
 */
public class HolaMundo {

    /**
     * Punto de entrada a la aplicación.
     * Este método imprime el mensaje "Hola mundo!"
     *
     * @param args Los argumentos de la línea de comando
     */
    public static void main(String[] args) {
        System.out.println("Hola mundo!");    // muestra el string por stdout
    }
}
```

Genera la documentación del programa mediante la sentencia:

¹ https://www.java.com/es/download/help/path_es.html

```
javadoc -author HolaMundo.java
```

Nota: es conveniente almacenar los ficheros de documentación en un directorio separado. Por ejemplo, *javadoc -d doc HolaMundo.java* los almacena en el directorio *doc*, creando el directorio si no existe.

Abre la página *index.html* para ver la documentación generada. Tienes más información sobre el formato adecuado para comentarios *JavaDoc* en: <http://en.wikipedia.org/wiki/Javadoc>

Apartado 3: Uso de librerías básicas (1 punto)

El siguiente programa calcula las longitudes de palabras proporcionadas por la línea de comandos.

```
import java.util.*;

/**
 * Esta clase calcula la longitudes de palabras y las almacena en un mapa.
 */
public class LongitudPalabras {
    private Map<String, Integer> palabras = new LinkedHashMap<>();
    /**
     * Constructor con las palabras.
     * @param palabras Palabras de las que se quiere calcular las longitudes.
     */
    public LongitudPalabras(String[] palabras) {
        for (String palabra: palabras)
            this.palabras.put(palabra, palabra.length());
    }
    /**
     * @return conjunto de palabras incluidas en el objeto.
     */
    public Set<String> getPalabras() {
        return this.palabras.keySet();
    }
    /**
     * @return conjunto de longitudes de las palabras.
     */
    public Collection<Integer> getLongitudes() {
        return this.palabras.values();
    }
    /**
     * Devuelve la longitud de una palabra, o -1 si no existe
     * @param palabra
     * @return su longitud
     */
    public int longitud(String palabra) {
        if (!this.palabras.containsKey(palabra)) return -1;
        return this.palabras.get(palabra);
    }
    /**
     * @return cadena de texto que representa este objeto.
     */
    @Override
    public String toString() {
        String str = "Las longitudes de las palabras son: \n";

        for (String palabra : this.palabras.keySet())
            str += "- " + palabra + " (" + this.palabras.get(palabra) + " caracteres).\n";

        return str;
    }
    /**
     * Punto de entrada a la aplicación.
     * Este método imprime las longitudes de palabras proporcionadas por la línea de comandos
     * @param args Los argumentos de la línea de comando. Se esperan palabras, como cadenas
     */
    public static void main(String[] args) {
        if (args.length == 0)
            System.err.println("Se espera al menos una palabra como parametro.");
        else {
            LongitudPalabras palabras = new LongitudPalabras(args);
            System.out.println(palabras);
            System.out.println("Palabras almacenadas: " + palabras.getPalabras());
            System.out.println("Longitud de 'No_almacenada': " + palabras.longitud("No_almacenada"));
        }
    }
}
```

El programa declara una clase `LongitudPalabras`. Las variables de esta clase (llamadas *objetos*), se crean llamando al método `public LongitudPalabras(String[] palabras)`. En programación orientada a objetos, este método se llama “*constructor*”, y en este caso recibe un parámetro de tipo array de `String`. La clase también define 4 métodos (`getPalabras`, `getLongitudes`, `longitud` y `toString`) que devuelven las palabras que se han incluido en el objeto, la longitud de todas las palabras, la longitud de una palabra concreta, y una representación del objeto en forma de cadena, para imprimirlo por pantalla. El punto de entrada es el método “*main*”, como en “C”. Como puedes observar, la llamada al constructor se realiza con el operador “*new*”, y la llamada a otros métodos del objeto se realiza con la notación “*objeto.metodo(params)*”.

Recuerda grabar el programa en un fichero llamado *LongitudPalabras.java*.

Algunas clarificaciones adicionales

- La referencia “*this*” es una constante que representa el objeto que ejecuta el método.
- En Java, para recorrer objetos puede usarse el “*for*” tradicional. No obstante, es mucho más frecuente y conveniente usar el “*for*” mejorado de la forma: `for (<Tipo> s: <coleccion>)` donde *s* es una variable de tipo `<Tipo>`, que va tomando cada uno de los valores de la colección `<coleccion>` en las iteraciones del bucle. Por ejemplo, el programa itera de esta manera sobre el array *palabras* dentro del constructor.
- Para la concatenación de cadenas se puede utilizar el operador “+”. Dicho operador puede usarse con cualquier tipo de dato, y este se convertirá automáticamente a cadena si cualquiera de los operandos es una cadena. Esta conversión automática también ocurre cuando ejecutamos `System.out.println(objeto)`. En este caso se llama automáticamente al método `toString()` del objeto. Como ves, el programa sobrescribe (*override*) el método `toString` para implementar el comportamiento específico para los objetos de tipo `LongitudPalabras`.
- En Java existe una librería estándar de utilidad (dentro del paquete `java.util` que se importa en la primera línea), con clases que implementan listas, conjuntos y mapas. El programa utiliza un tipo de mapa (`LinkedHashMap`) que guarda las claves por orden de inserción. Los mapas disponen de métodos como `get` (para obtener el valor asociado a una clave), `put` (para almacenar una clave), o `keySet` (que devuelve el conjunto de claves, y que usamos para iterar el mapa en el método `toString`).

Contesta a las siguientes preguntas:

Ejecuta el programa con distintos tipos de parámetros, sin parámetros, o con más de uno. ¿Qué sucede?

Apartado 4: Tu primer programa Java (9 puntos)

Basándote en el programa anterior, crea un programa Java que devuelva la *frecuencia* de cada longitud de palabra. Es decir, el programa devolverá el número de palabras que tienen la misma longitud. Para ello, imprimirá por cada una de las longitudes de palabras almacenadas, la frecuencia de cada una de ellas. Por ejemplo, para las palabras: *codigo*, *casa* y *gato*, el programa deberá indicar las siguientes frecuencias: *1 palabra con 6 letras y 2 palabras con 4 letras*.

Completa el siguiente programa, añadiendo métodos a la clase `LongitudPalabras`, para obtener la salida de más abajo:

```
public class FrecuenciaPalabras {
    public static void main(String[] args) {
        if (args.length == 0)
            System.err.println("Se espera al menos una palabra como parametro.");
        else {
            LongitudPalabras palabras = new LongitudPalabras(args);
            imprimeFrecuencias(palabras);
        }
    }

    private static void imprimeFrecuencias(LongitudPalabras palabras) {
        System.out.println(palabras);

        for(int longitud : palabras.getLongitudesUnicas())
            System.out.println("Hay "+palabras.getFrecuencia(longitud)+" palabras de "+longitud+" letras.");
    }
}
```

Si ejecutamos el programa con las palabras *codigo casa gato* obtendremos la siguiente salida:

Las longitudes de las palabras son:

- codigo (6 caracteres).
- casa (4 caracteres).
- gato (4 caracteres).

Hay 2 palabras de 4 letras.

Hay 1 palabras de 6 letras.

Algunas indicaciones (aunque trataremos todas estas cuestiones en mucho mayor detalle durante el curso):

- La liberación de memoria en Java es automática, como veremos en los próximos temas, por lo que no necesitas liberar memoria ni destruir objetos al final del programa.
- Las variables internas de una clase (llamadas atributos) pueden declararse con un control de acceso (`private`, `public` y otros). Normalmente serán de tipo `private` para evitar que sean modificadas desde el exterior. También puede declararse un control de acceso para los métodos: los privados serán las funciones auxiliares (sólo necesarias internamente, desde dentro de la clase), y los públicos serán los que se pueden llamar desde fuera de la clase. Evita la repetición de código en los métodos que añadas, usando métodos auxiliares en caso necesario.
- Un método con modificador `static` (como `imprimeFrecuencias`) es un método de clase, y no necesita el contexto de un objeto para ejecutarse. Por el contrario, sin `static`, el método necesita el contexto de un objeto para ejecutarse (todos los métodos de `LongitudPalabras` del apartado 3 son métodos de objeto).
- El código del programa debes incluirlo en un archivo llamado *FrecuenciaPalabras.java*. Como has observado, puedes usar la clase `LongitudPalabras` desde la clase *FrecuenciaPalabras*, ya que están en el mismo paquete (pero aprenderemos más sobre paquetes más adelante).
- Para poder implementar el método `getLongitudesUnicas`, uno de los aspectos que debes tener en cuenta es evitar las repeticiones de frecuencias. Para lograrlo puedes utilizar un conjunto (`HashSet`) para almacenar las longitudes de las palabras. Los conjuntos no permiten repetición.
- Durante el curso, veremos que el nombre de las clases en Java suelen ser sustantivos en singular.

Contesta a las siguientes preguntas:

- ¿Qué estrategia has seguido para el cálculo de las frecuencias? ¿Qué otras alternativas se te ocurren para ese cálculo? Indica sus ventajas e inconvenientes.
- ¿Cómo implementarías un método `getLongitudesRepetidas` que devuelva las longitudes de las palabras que se repiten al menos una vez? Con las palabras *codigo casa gato*, la salida debería ser únicamente la longitud 4 letras, que es la única que se repite.

Normas de entrega:

- El nombre de los alumnos debe ir en la cabecera *JavaDoc* de todas las clases entregadas
- La entrega la realizará uno de los alumnos de la pareja a través de Moodle
- Se debe entregar un único fichero ZIP / RAR con todo lo solicitado, que deberá llamarse de la siguiente manera: GR<numero_grupo>_<nombre_estudiantes>.zip. Por ejemplo Marisa y Pedro, del grupo 2261, entregarían el fichero: GR2261_MarisaPedro.zip
- La estructura de los ficheros entregados deberá ser la siguiente:
 - **src**. Ficheros fuente
 - **doc**. Documentación *JavaDoc* generada
 - **cuestiones.txt**. Respuestas a las preguntas planteadas en los apartados 3 y 4.